AWS VPC Peering Lab

Field	Value
Student name	Gautam
Batch	1
Region	Kurukshetra, Haryana, India
Completion date	29/10/2026

Project summary: This project builds two AWS VPCs connected via a peering connection. VPC A hosts a public bastion host and a private application server (App-A). VPC B hosts a second private application server (App-B) with no internet gateway. VPC peering allows App-A and App-B to exchange traffic over private IPv4 addresses, while all administrative access flows through the bastion using SSH ProxyJump.

1. Architecture & Resource Inventory

One internet entry point, two private workloads, and a direct peering connection between their VPCs.

Resource / Name tag	VPC or subnet	IPv4 CIDR / address	Actual resource ID
ewb-vpc-a	VPC A	10.10.0.0/16	vpc-0a7f3c9e2b6d4185f
ewb-a-public	VPC A / public subnet	10.10.1.0/24	subnet-0b8e4d7f1a2c69350
ewb-a-private	VPC A / private subnet	10.10.2.0/24	subnet-0c9d2e5f7a1b63840
ewb-vpc-b	VPC B	10.20.0.0/16	vpc-07d2e8a4c1f9635b0
ewb-b-private	VPC B / private subnet	10.20.1.0/24	subnet-06f3a9c2d8e1474b0
ewb-bastion	ewb-a-public	10.10.1.10 + public IPv4	i-0c8e5f2a9d7143b60
ewb-app-a	ewb-a-private	10.10.2.10 / no public IP	i-03a7c91e5b2d846f0
ewb-app-b	ewb-b-private	10.20.1.10 / no public IP	i-07d4be9c6a1835f0

Other named resources: Internet gateway **ewb-igw-a** · Route tables **ewb-rt-a-public**, **ewb-rt-a-private**, **ewb-rt-b-private** · Security groups **ewb-sg-bastion**, **ewb-sg-app-a**, **ewb-sg-app-b** · Peering connection **ewb-a-b-peer** · EC2 key pair **ewb-lab**.

2.Subnets, Internet Gateway & Route Tables

ewb-rt-a-public (associated to ewb-a-public)

Destination	Target	Purpose
10.10.0.0/16	local	Bastion to App-A
0.0.0.0/0	ewb-igw-a	Bastion internet access
10.20.0.0/16	pcx-0d5274dedfee06e15	Bastion SSH to App-B

ewb-rt-a-private (associated to ewb-a-private)

Destination	Target	Purpose
10.10.0.0/16	local	Existing VPC A route
10.20.0.0/16	pcx-0d5274dedfee06e15	App-A to VPC B

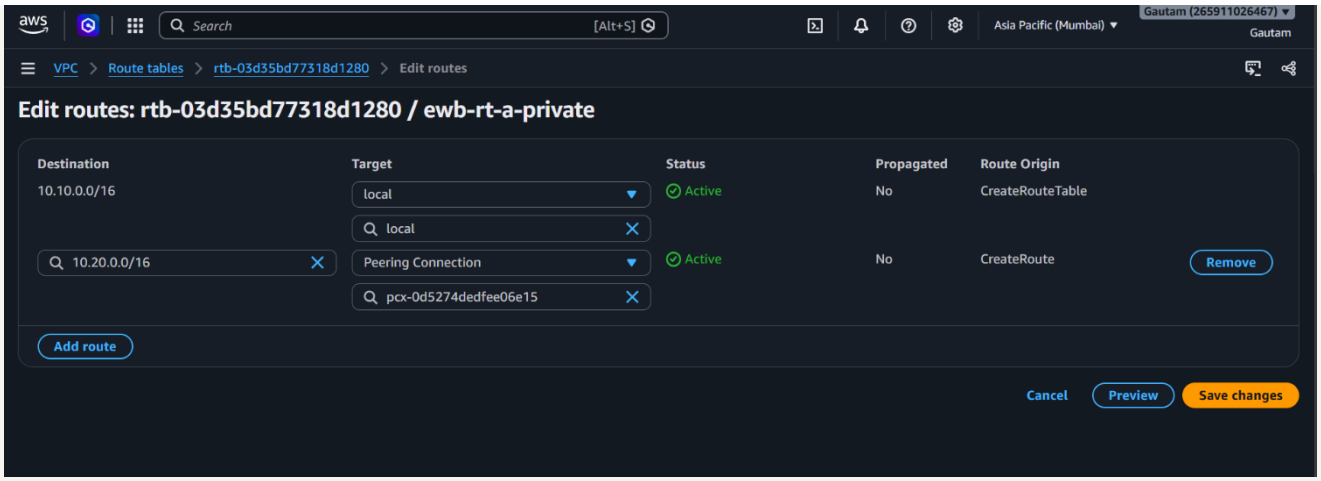
ewb-rt-b-private (associated to ewb-b-private)

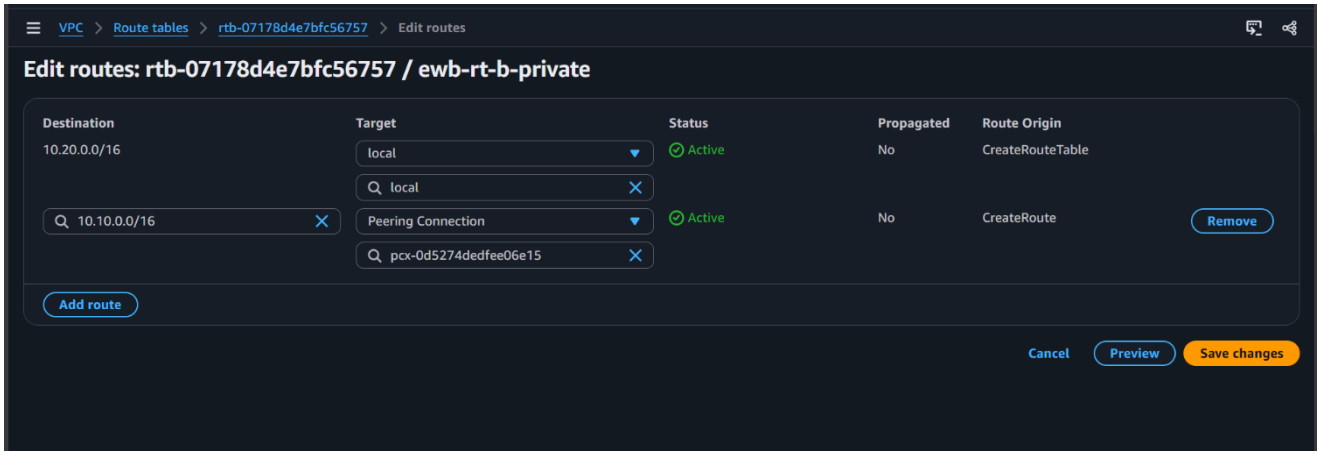
Destination	Target	Purpose
10.20.0.0/16	local	Existing VPC B route
10.10.0.0/16	pcx-0d5274dedfee06e15	Return path to VPC A

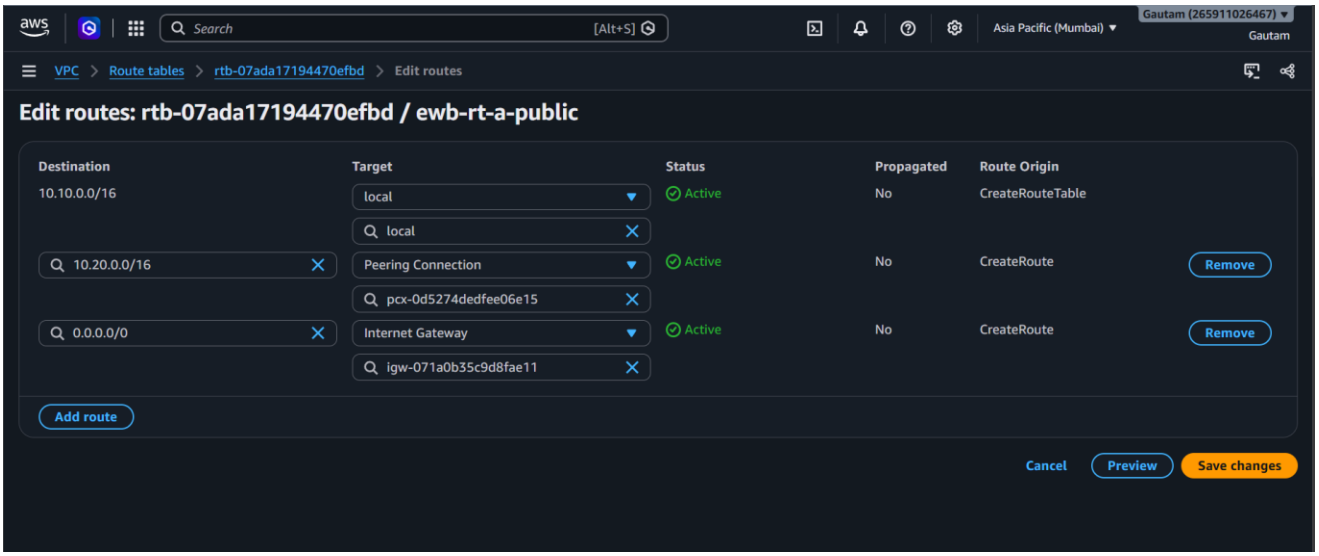
3.Security Groups & Peering Connection

Inbound security group rules

Security group	Type / port	Source
ewb-sg-bastion (VPC A)	SSH / TCP 22	Your current public IPv4 /32
ewb-sg-app-a (VPC A)	SSH / TCP 22	10.10.1.10/32 (bastion)
ewb-sg-app-a (VPC A)	Custom ICMP IPv4, Echo request	10.20.1.10/32 (App-B)
ewb-sg-app-b (VPC B)	SSH / TCP 22	10.10.1.10/32 (bastion)
ewb-sg-app-b (VPC B)	Custom ICMP IPv4, Echo request	10.10.2.10/32 (App-A)

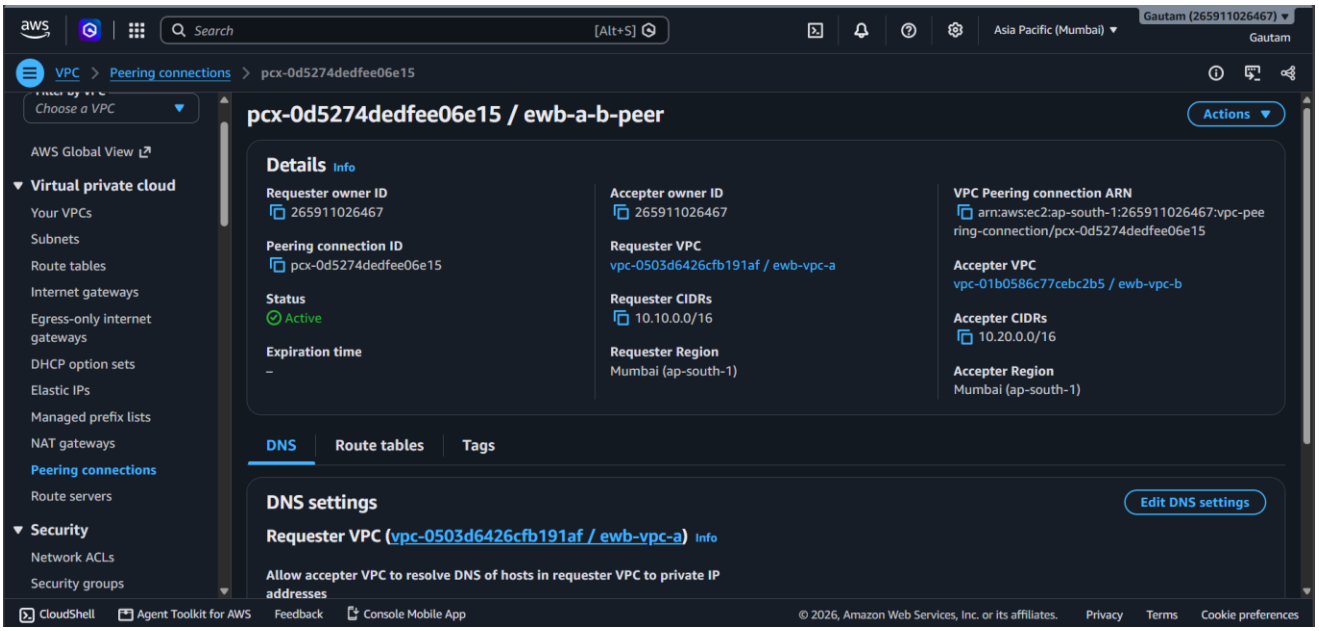

Search
[Alt+S]
Asia Pacific (Mumbai)
Gautam (265911026467)
Gautam
VPC > Route tables > rtb-03d35bd77318d1280 > Edit routes
Edit routes: rtb-03d35bd77318d1280 / ewb-rt-a-private
Destination
Target
Status
Propagated
Route Origin
10.10.0.0/16
local
Active
No
CreateRouteTable
local
10.20.0.0/16
Peering Connection
Active
No
CreateRoute
Remove
pcx-0d5274dedfee06e15
Add route
Cancel
Preview
Save changes


VPC > Route tables > rtb-07178d4e7bfc56757 > Edit routes
Edit routes: rtb-07178d4e7bfc56757 / ewb-rt-b-private
Destination
Target
Status
Propagated
Route Origin
10.20.0.0/16
local
Active
No
CreateRouteTable
local
10.10.0.0/16
Peering Connection
Active
No
CreateRoute
Remove
pcx-0d5274dedfee06e15
Add route
Cancel
Preview
Save changes



Peering connection

Name: **ewb-a-b-peer** · Requester: ewb-vpc-a (10.10.0.0/16) · Accepter: ewb-vpc-b (10.20.0.0/16) · Status: **Active** · Peering ID: pcx-0d5274dedfee06e15



4.Test Evidence

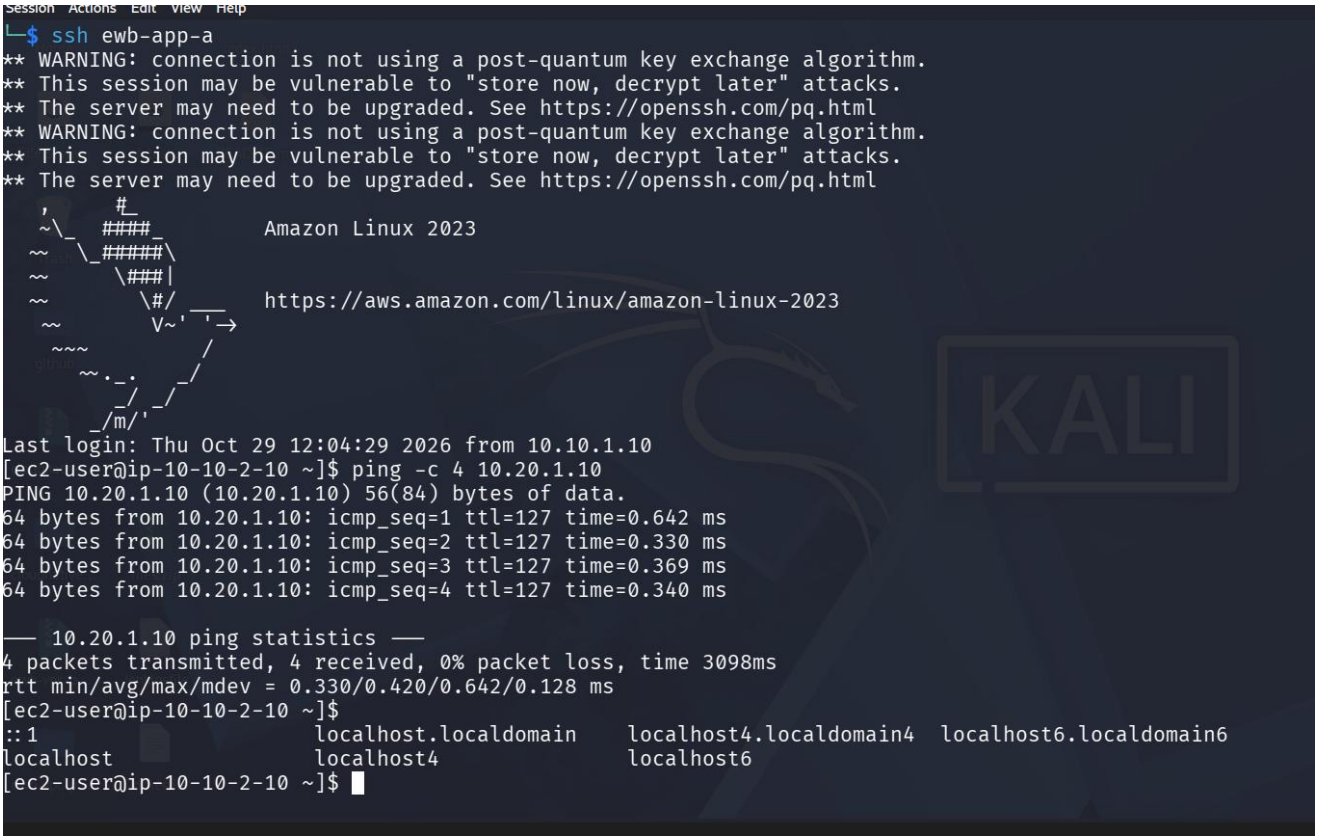
Test 01 — Baseline (before peering)

Where: App-A, before peering · **Command:** `ping -c 4 10.20.1.10` · **Expected:** No replies

```
/home/ec2-user
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3081ms
```

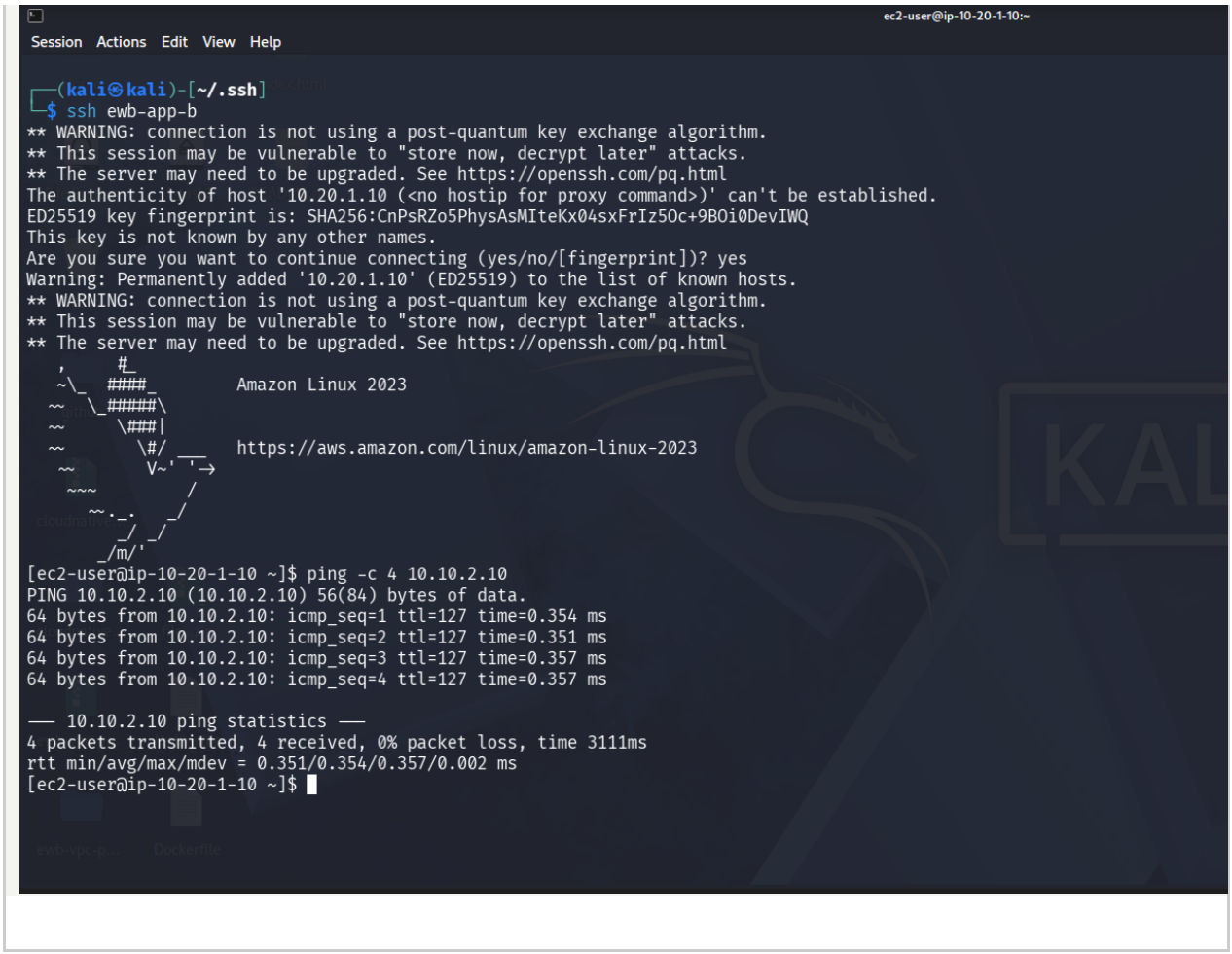
Test 02 — Forward path (App-A → App-B, after routes)



Where: App-A, after routes · **Command:** `ping -c 4 10.20.1.10` · **Expected:** 4 replies, 0% packet loss

Test 03 — Reverse path (App-B → App-A, after routes)

Where: App-B, after routes · **Command:** `ping -c 4 10.10.2.10` · **Expected:** 4 replies, 0% packet loss



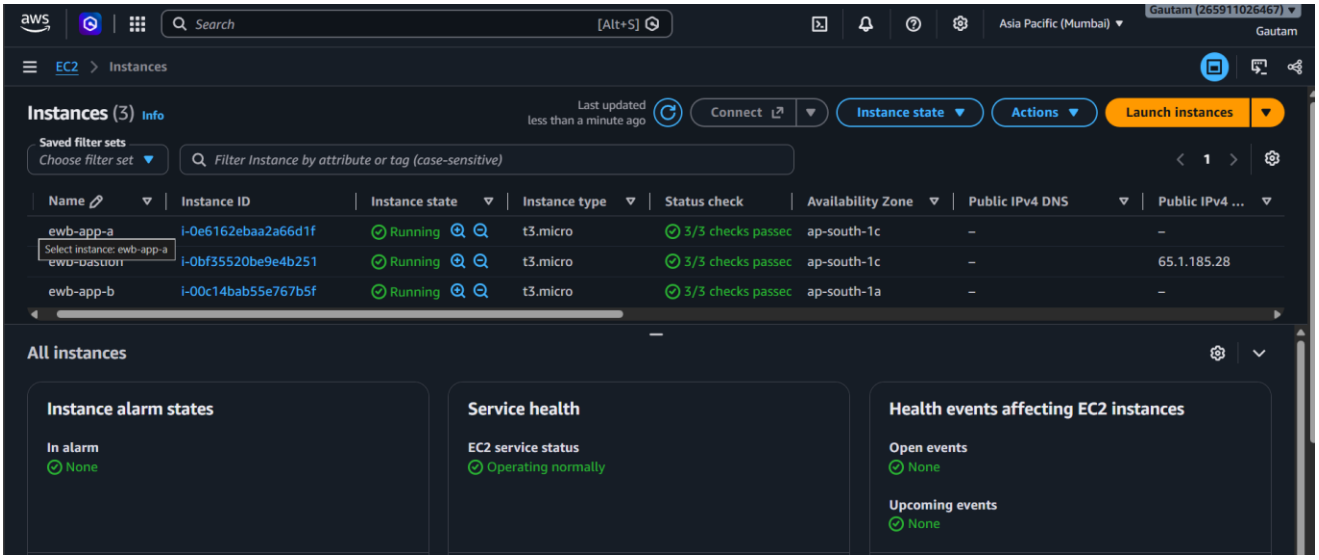
Test 04 — Private SSH (laptop → App-B via bastion)

Where: Laptop · **Command:** `ssh ewb-app-b` · **Expected:** Shell on App-B through the bastion


ec2-user@ip-10-20-1-10:~
Session Actions Edit View Help
┌──(kali㉿kali)-[~/.ssh]
└─$ ssh ewb-app-b
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
The authenticity of host '10.20.1.10 (<no hostip for proxy command>)' can't be established.
ED25519 key fingerprint is: SHA256:CnPsRZo5PhysAsMIteKx04sxFrIz5Oc+9BOi0DevIWQ
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.20.1.10' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023
[ec2-user@ip-10-20-1-10 ~]$ ping -c 4 10.10.2.10
PING 10.10.2.10 (10.10.2.10) 56(84) bytes of data.
64 bytes from 10.10.2.10: icmp_seq=1 ttl=127 time=0.354 ms
64 bytes from 10.10.2.10: icmp_seq=2 ttl=127 time=0.351 ms
64 bytes from 10.10.2.10: icmp_seq=3 ttl=127 time=0.357 ms
64 bytes from 10.10.2.10: icmp_seq=4 ttl=127 time=0.357 ms
— 10.10.2.10 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3111ms
rtt min/avg/max/mdev = 0.351/0.354/0.357/0.002 ms
[ec2-user@ip-10-20-1-10 ~]$

Test 05 — Isolation (no public IPv4)

Where: EC2 instance details for App-A and App-B · **Expected:** Public IPv4 address field is blank for both



2. Controlled Failure & Recovery

Temporarily removed the App-B inbound ICMP Echo request rule (source 10.10.2.10/32) from ewb-sg-app-b, re-ran the forward ping from App-A, restored the exact rule, and re-ran the test.

Explanation: This confirms the security group rule — not just the route — controls whether ICMP traffic is permitted between App-A and App-B. The peering connection and route tables remained Active and unchanged throughout; only the security group rule was modified, proving that connectivity depends on both routing and security group configuration together.

3. Forward Path & Return Path Explanation

Forward path: App-A (10.10.2.10, VPC A) sends traffic to App-B (10.20.1.10, VPC B). The private route table ewb-rt-a-private has a route for 10.20.0.0/16 pointing to the peering connection (pcx-ID), so traffic leaves VPC A through the peering link instead of being dropped.

Return path: For App-B's reply to reach back to App-A, VPC B's route table (ewb-rt-b-private) needs its own route for 10.10.0.0/16 pointing to the same peering connection. Without this return route, the forward ping would leave VPC A successfully but the reply would never make it back — resulting in 100% packet loss even with one-directional routing in place.

Why the public subnet also needs a peering route: When SSHing to App-B via `ssh ewb-app-b`, the ProxyJump configuration makes the bastion itself open a direct connection to App-B's private IP. Since the bastion sits in ewb-a-public, that subnet's route table (ewb-rt-a-public) also needs the 10.20.0.0/16 route to pcx-ID — otherwise the bastion has no way to reach VPC B.

4. What we learned

AWS VPC Peering and Secure Network Connectivity
Designed and configured a two-VPC AWS network architecture using Amazon VPC, EC2, custom route tables, security groups, an Internet Gateway, and VPC peering. Implemented private communication between application servers, established secure SSH access through a bastion host using ProxyJump, and performed connectivity testing, controlled failure testing, troubleshooting, and resource cleanup.